实验二 文档倒排索引算法实现

姓名：张瑞泽　学号：202400390037　班级：一班

日期：2026 年 10 月 28 日　Email：1639821161@qq.com

GitHub 公开仓库：https://github.com/1-ze/hadoop-inverted-index-lab2

一 实验目的

本实验使用 Hadoop MapReduce 对公开英文文档建立倒排索引，掌握从原始文本处理、HDFS 上传到 YARN 作业提交和结果验证的完整过程。索引不仅统计一个词出现了多少次，还记录这些次数分别来自哪些文档。

我以 12 篇 Apache Hadoop 技术文档为输入，建立停词表，通过 Mapper 提取词项、Reducer 汇总文档词频，并用独立 Python 程序对全部输出进行核对。

二 实验软件和硬件环境

项目	实际环境
虚拟化与操作系统	VirtualBox；Ubuntu 22.04.5 LTS
计算资源	4 GB 内存；2 个虚拟 CPU
Java 与 Hadoop	OpenJDK 11.0.32；Hadoop 3.4.3
部署方式	单机伪分布式 HDFS、YARN、MapReduce
辅助工具	Python 3 标准库；UTF-8 文本

本次重新编译并运行索引程序；实验零的 Hadoop 环境用于提供基础服务。语料、索引输出、日志和校验结果均保存在本次实验目录。

三 文档数据及来源

数据来自 Apache Hadoop 3.4.3 官方公开技术文档。下载后仅提取网页主体文本，删除导航、脚本、样式和多余空白，保留正文内目录、命令与配置示例。每个网页对应一个 UTF-8 文件，避免将网页组件重复计入语料。

文档文件	主题	原始词次
document01.txt	单节点安装	942
document02.txt	集群部署	3,044
document03.txt	HDFS 架构	4,401
document04.txt	HDFS 用户指南	3,854
document05.txt	文件系统 Shell	6,053
document06.txt	MapReduce 教程	8,721
document07.txt	YARN 架构	494
document08.txt	Hadoop Streaming	4,204
document09.txt	Hadoop 归档	840
document10.txt	DistCp 工具	5,755
document11.txt	HDFS 快照	1,169
document12.txt	HDFS 高可用 QJM	6,479

共 12 篇文档，原始英文词次为 45,956。全部来源网址、下载时间及 SHA-256 校验值保存在 data/sources.json，便于确认输入版本。

来源地址前缀：https://hadoop.apache.org/docs/r3.4.3/。网页资料采用 Apache License 2.0，仓库保留 LICENSE、NOTICE 及原始来源说明。

四 停词表与实验原理

停词表用于过滤频繁出现而检索区分度较低的功能词。本实验使用以下 46 个词；不额外过滤 data、hadoop 等与文档主题有关的单词。完整内容与仓库 stopwords.txt 一致。

a，an，and，are，as，at，be，been，but，by，for，from，has，have，he，her，his，i，if，in，is，it，its，not，of，on，or，our，she，that，the，their，them，there，these，they，this，to，was，we，were，which，will，with，you，your

文本处理规则

先用正则表达式 [A-Za-z]+ 匹配连续英文字母，再统一转为小写并过滤停词。数字、标点和连字符均作为分隔符，例如 Data 与 data 合并，index-index 拆为两个 index。没有采用词干提取，file 与 files 分别统计。

MapReduce 数据流

英文文档 → Mapper 分词与停词过滤 → 按单词 Shuffle 分组 → Reducer 累计各文档词频 → 输出倒排列表及总次数。

Mapper 从 FileSplit 中获取输入文件名，对每个保留词输出〈单词，文档名〉。Shuffle 将同一单词的所有文档名聚合到一个组；Reducer 使用 TreeMap 按文件名累计次数，并同时计算该词在整个语料中的总次数。

最终结果中的每一项〈文档名，次数〉表示该词在这一文档中的词频；〈total，次数〉等于前面所有文档词频之和。本实验要求的是文档词频索引，没有记录词在文档中的具体偏移位置。

考虑到只有 12 篇文档，本实验设置一个 Reducer，使全部词项按字典序输出。该设计便于结果检查，但大规模语料需要更多 Reducer 分摊负载；此时单个输出分片有序，多个分片不再天然构成一个全局有序文件。

五 关键伪代码与程序构建

```
Mapper.setup:
  从缓存文件加载停词表
  document ← 当前输入文件名
Mapper.map(line):
  对每个匹配 [A-Za-z]+ 的 word:
    word ← 转小写(word)
    如果 word 不在停词表: emit(word, document)
Reducer.reduce(word, documents):
  counts ← 按文档名排序的映射; total ← 0
  对每个 document:
    counts[document] += 1; total += 1
  输出 word、文档词频列表和 total
```

停词表通过 Job.addCacheFile 分发到各 Mapper。Reducer 保存文档名时调用 toString() 复制内容，避免 Hadoop 复用 Text 对象引起统计错误。最终格式化的 Reducer 没有直接配置为 Combiner。

构建命令：bash scripts/build.sh。脚本使用 hadoop classpath 获取依赖，通过 javac 编译 Java 11 目标代码，再打包生成 inverted-index.jar。

六 Hadoop 作业执行过程

1. 启动 HDFS 与 YARN，通过 jps 检查守护进程，通过 hdfs dfsadmin -report 确认 DataNode 可用。沿用已有文件系统，不重新格式化 NameNode。

2. 运行 bash scripts/run.sh real_20261028，创建本次独立的 HDFS 输入目录，上传 12 篇文本和停词表。脚本在目标目录已存在时退出，避免覆盖之前实验。

3. 提交 JAR 作业，检查执行进度与计数器，导出 part-r-00000；再核对 YARN 的最终状态和 HDFS 输出目录。

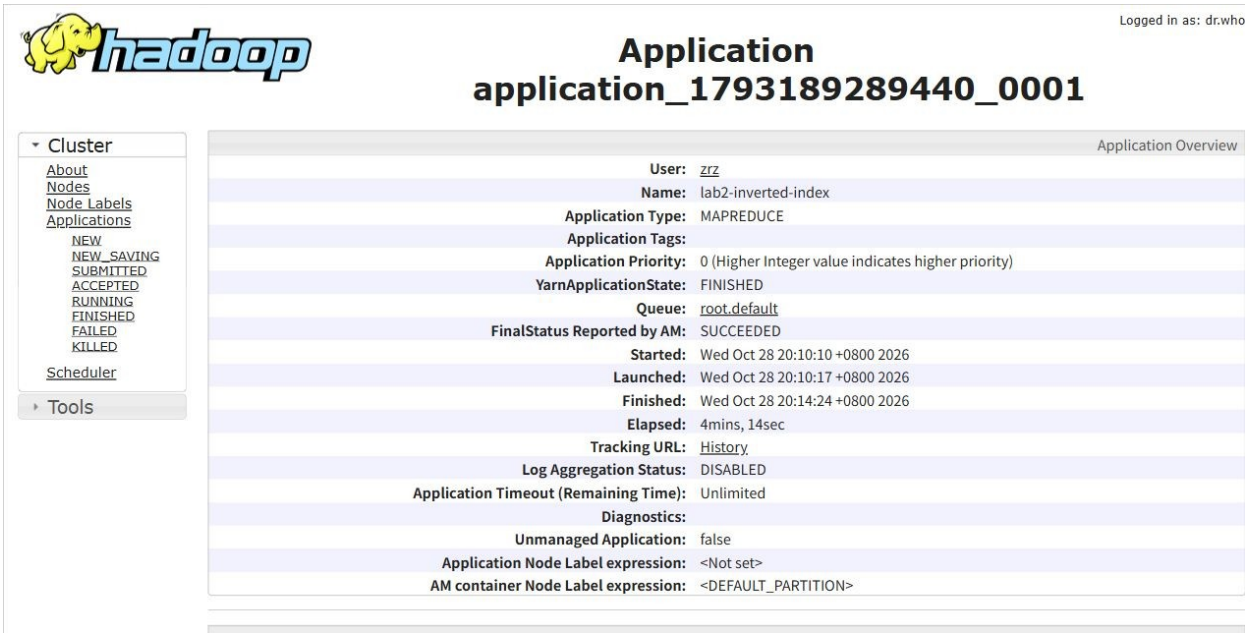


图 1 YARN 中本次作业的最终状态

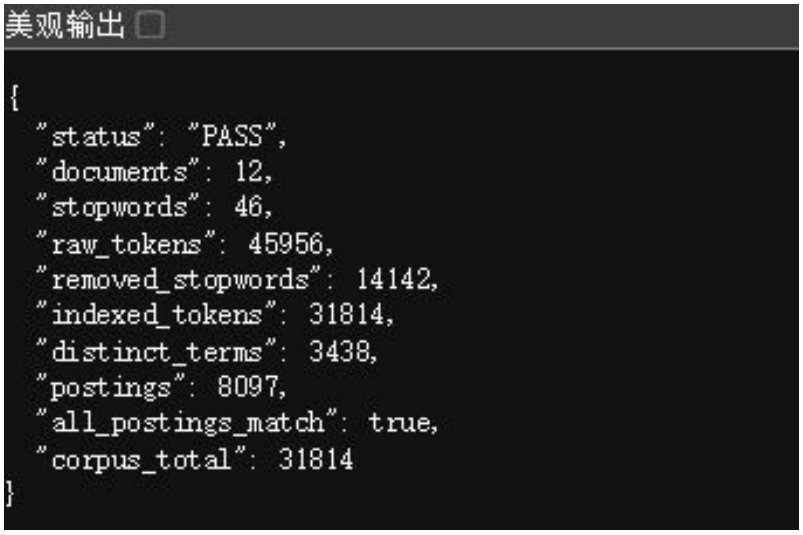


图 2 独立全量校验结果文件

七 实验结果与独立校验

统计项目	实际结果
输入文档数	12
原始英文词次	45,956
过滤停词次数	14,142
进入索引的词次	31,814
不同词项数	3,438
词项与文档关联数	8,097

代表词	涉及文档数	总次数
data	12	159
hadoop	11	611
index	2	3
mapreduce	9	203
search	2	3

index <document05.txt,1>;<document09.txt,2>;<total,3>.

search <document03.txt,1>;<document06.txt,2>;<total,3>.

独立校验器重新扫描全部输入，使用 Python Counter 生成参考结果，再逐项比较词项、文档名和出现次数。全部倒排项一致；每个词的 total 均等于其文档词频之和，全部 total 之和等于进入索引的词次。

另一次实际 Hadoop 作业验证大小写归一、重复词、标点、数字、空文件和仅含停词的文件。人工可计算的预期结果中，data=4、hadoop=2、index=2、search=1，作业输出与预期一致。

完整输出见 results/inverted-index.txt；全量校验见 validation.json；边界结果见 edge-tests.txt。计数器与这些独立统计相互核对，避免仅凭程序成功退出就认定索引正确。

八 结论分析与问题解答

本实验完成了 12 篇英文文档的倒排索引，过滤停词后保留 31,814 个词次，形成 3,438 个词项和 8,097 个词项与文档关联。完整语料重新运行得到的 Hadoop 输出通过独立全量比对，边界作业也与人工预期一致。

与 WordCount 相比，倒排索引需要在单词频次之外保留文档来源。实验中，文件名的提取、停词表在任务间的分发，以及 Reducer 中可变对象的处理直接影响结果。公开仓库中的脚本和固定语料使这一过程能够重新执行和检查。

问题一 为什么需要独立校验程序

Hadoop 显示 SUCCEEDED 只说明任务执行完成，不能保证分词和统计逻辑符合要求。用独立实现遍历所有文档，再比较全部倒排项，可以发现遗漏文档、重复计数及 total 错误；小样本边界作业则提供容易人工核对的结果。

问题二 为什么不能直接复用最终 Reducer 作为 Combiner

本程序 Mapper 输出文档名，而最终 Reducer 输出已格式化的列表，后者不再是 Reducer 所要求的文档名输入。若要增加 Combiner，应先设计可重复合并的结构，例如〈文档名，局部次数〉，并同时调整 Reducer，不能只添加一个配置项。

问题三 如何处理再次运行时输出目录已存在

Hadoop 通常拒绝写入已存在的输出目录。本实验为每次运行创建新的目录，并在路径已存在时退出。这样可以保留历史结果，也避免为方便运行而误删其他实验的数据。

参考资料与代码

Apache Hadoop 3.4.3 MapReduce Tutorial 与 HDFS、YARN 官方文档。各篇精确网址见仓库 data/sources.json。

代码与报告：https://github.com/1-ze/hadoop-inverted-index-lab2